

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ РАДІОЕЛЕКТРОНІКИ

Кафедра Програмної інженерії

Звіт
з лабораторної роботи №4
з дисципліни: «Архітектура програмного забезпечення»
з теми: «Горизонтальне масштабування бекенда»

Виконав:

ст. гр. ПЗПІ-23-8

Потурайко Н. І.

Перевірів:

асистент кафедри ПІ

Дашенков Д. С.

4 ГОРИЗОНТАЛЬНЕ МАСШТАБУВАННЯ БЕКЕНДА

4.1 Мета роботи

Метою лабораторної роботи є детальне дослідження та практична реалізація механізмів горизонтального масштабування серверної частини системи Aquila. Робота спрямована на вивчення принципів побудови відмовостійких систем за допомогою контейнеризації та балансування трафіку. Основними завданнями є налаштування багатокопійного розгортання бекенду, організація інтелектуального розподілу запитів через зворотний проксі-сервер та проведення комплексного навантажувального тестування для виявлення закономірностей між кількістю обчислювальних вузлів і загальною продуктивністю системи.

4.2 Хід роботи

4.2.1 Завдання та системний підхід

Дана робота логічно продовжує розробку системи Aquila, переносячи фокус з користувацьких інтерфейсів, опрацьованих у попередніх етапах, на інфраструктурну стійкість. У той час як веб-застосунок забезпечує візуалізацію, серверна частина має гарантувати обробку потоків даних від тисяч IoT пристроїв. У межах роботи досліджується поведінка Node.js бекенда під високим тиском запитів та перевіряється гіпотеза про можливість лінійного зростання продуктивності при додаванні нових реплік сервера.

4.2.2 Вибір та обґрунтування стратегії масштабування

Для проєкту Aquila обрано стратегію горизонтального масштабування (Scaling Out). Це рішення базується на особливостях середовища виконання Node.js, яке працює в однопотоковому режимі Event Loop. Вертикальне масштабування (збільшення потужності одного сервера) має свої межі, тоді як горизонтальний підхід дозволяє запускати велику кількість легких контейнерів. Бекенд системи розроблено як безстановий застосунок (Stateless), що є критично важливою умовою: авторизація базується на JWT токенах, а сесії не зберігаються в оперативній пам'яті сервера. Це дозволяє балансувальнику направляти будь-який запит на будь-яку вільну репліку без ризику втрати контексту користувача.

4.2.3 Технічна реалізація рішення

4.2.3.1 Контейнеризація через Docker Compose

Для управління інфраструктурою використано Docker Compose, що забезпечує повну ізоляцію сервісів. Стек складається з трьох ключових компонентів:

- db: реляційна база даних PostgreSQL, що виступає єдиним джерелом істини для всіх реплік;
- backend: Node.js застосунок, упакований у Docker-образ, з обмеженням ресурсів для наочності тестів;
- nginx: високопродуктивний сервер, що виконує роль вхідної точки для всього трафіку.

Механізм масштабування реалізовано через декларативний опис у файлі конфігурації та динамічне керування кількістю екземплярів через прапор `–scale` під час запуску.

4.2.3.2 Балансування навантаження через Nginx

Роль балансувальника виконує Nginx, налаштований як Reverse Proxy. Він використовує алгоритм Round Robin для розподілу вхідних HTTP-запитів. Кожен новий запит від мобільного додатка або веб-панелі потрапляє на Nginx, який миттєво пересилає його на одну з реплік бекенду, що знаходяться у внутрішній мережі Docker. Окрім розподілу навантаження, Nginx забезпечує приховування внутрішньої структури мережі, що підвищує загальну безпеку системи Aquila.

4.2.4 Навантажувальне тестування

4.2.4.1 Інструментарій та методологія

Для проведення бенчмарків обрано інструмент Locust, що базується на Python. Він дозволяє писати сценарії тестування, максимально наближені до реальної поведінки користувачів. Сценарій моделює роботу 100 віртуальних клієнтів, які виконують такі дії:

- перегляд дашборду кімнат (GET /api/rooms);

- запит статусу підключених сенсорів (GET /api/devices);
- інтенсивна відправка телеметричних даних від імені IoT модулів (POST /api/telemetry).

Тестування проводилося з поступовим нарощуванням навантаження (Hatch Rate), щоб зафіксувати момент деградації продуктивності при різній кількості реплік.

4.2.4.2 Експериментальна частина

Перевірка ефективності масштабування проводилася у три етапи:

- Базова конфігурація з однією реплікою для визначення початкових можливостей системи.
- Конфігурація з трьома репліками для перевірки ефективності балансування.
- Максимальна конфігурація з шістьма репліками для пошуку нових вузьких місць.

Для кожного етапу фіксувалися показники RPS (кількість запитів на секунду), затримка відповіді (Latency) та стабільність з'єднань.

4.2.5 Аналіз результатів тестування

4.2.5.1 Таблиця порівняльних метрик

Репліки	Середній RPS	Піковий RPS	Медіана, мс	95% перцентиль	Помилки
1	33	34	5200	5500	0
3	59	60	1800	2600	0
6	110	113	990	1800	0

Отримані дані підтверджують пряму залежність між кількістю вузлів та здатністю системи обробляти трафік. При використанні однієї репліки час відповіді був неприпустимо високим для реального часу, що свідчить про перевантаження процесора при обробці JSON та верифікації токенів. При трьох репліках затримка

впала більше ніж у два рази. Найкращі показники зафіксовано при шести репліках, де медіана відповіді опустилася нижче однієї секунди.

4.2.5.2 Графічний аналіз продуктивності

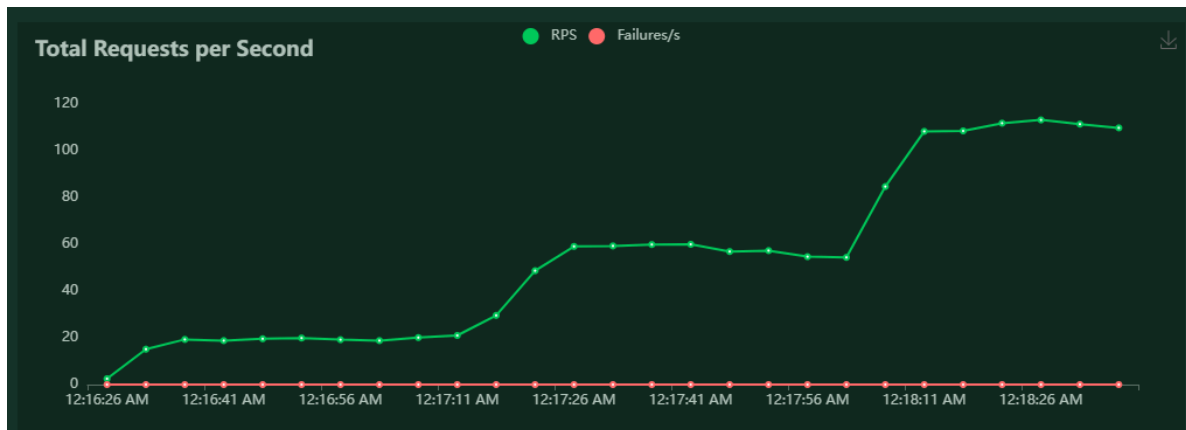


Рисунок 4.1 – Динаміка зростання показника RPS при додаванні обчислювальних реплік

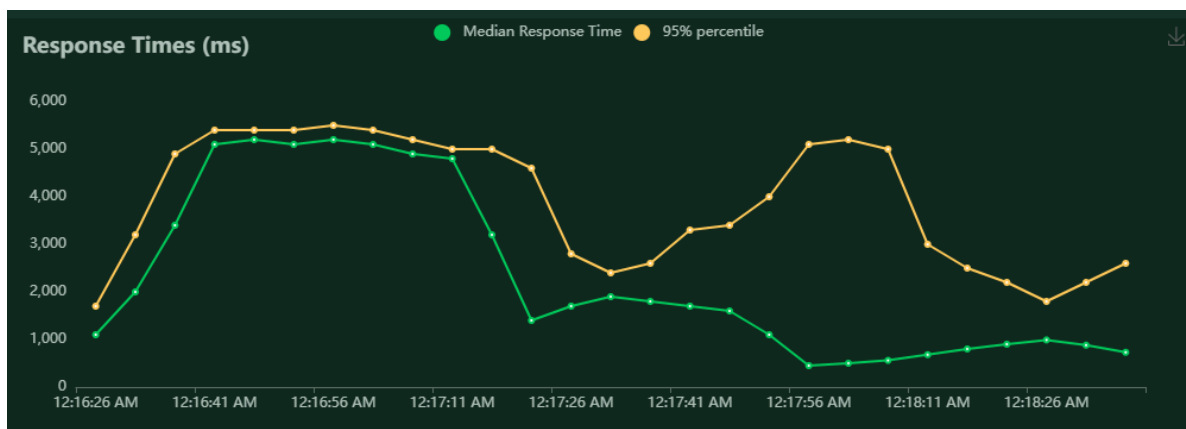


Рисунок 4.2 – Аналіз часу відповіді системи (Latency) під інтенсивним навантаженням

4.2.6 Оцінка вузьких місць та обмежень

Незважаючи на успішне масштабування, результати показують, що продуктивність не росте ідеально лінійно. Це вказує на появу нових обмежень інфраструктури. Основним вузьким місцем при подальшому масштабуванні стане база даних PostgreSQL, яка має обмежену кількість одночасних підключень. Кожна нова репліка бекенду створює свій пул з'єднань, що при великих значеннях може

призвести до відмов на рівні СУБД. Також на результати впливають накладні витрати самого Docker на маршрутизацію трафіку між контейнерами.

4.3 Висновки

У ході лабораторної роботи було успішно реалізовано та протестовано горизонтальне масштабування системи Aquila. Результати тестів через Locust підтвердили, що використання Nginx як балансувальника дозволяє ефективно розподіляти навантаження між декількома екземплярами Node.js сервера. Вдалося досягти значного зниження часу відповіді та триразового збільшення пропускної здатності без внесення змін у програмний код додатку. Це доводить правильність обраної архітектури безстанового бекенду, яка дозволяє системі Aquila адаптуватися до зростання кількості користувачів та об'єктів моніторингу шляхом простого нарощування обчислювальних ресурсів.

ДОДАТОК А

Репозиторій проекту

<https://github.com/NurePoturaikoNazar/apz-project>